Module 03 Lab - CNNs for Image Classification: Puppy or Bagel? 🐶🥯

Welcome to Module 03! In this lab, we will explore **Convolutional Neural Networks (CNNs)** for image classification using one of the internet's most amusing classification challenges: **Can you tell the difference between a puppy and a bagel?**

What you'll learn:

- CNN fundamentals: convolutional layers, pooling, and feature maps
- Building a CNN from scratch for binary image classification
- Transfer learning with pre-trained models
- Fine-tuning strategies for custom datasets

Dataset: Puppy or Bagel

- **Kaggle Link:** https://www.kaggle.com/datasets/returnofsputnik/puppy-or-bagel

The Famous Meme That Started It All!

This dataset is inspired by Karen Zack's (@teenybiscuit) viral meme showing how curled-up puppies look remarkably similar to bagels!

```
# ============================================================
# DISPLAY THE FAMOUS PUPPY OR BAGEL MEME
# This cell shows the viral meme that inspired our dataset
# ============================================================

from IPython.display import Image, display, HTML

# Display the original "Puppy or Bagel" meme by Karen Zack
display(HTML('''
<div style="text-align: center; padding: 20px; background-color: #f5f5f5; border-radius: 10px;">
    <h3>🐶 The Famous "Puppy or Bagel?" Challenge 🥯</h3>
    <img src="https://static.boredpanda.com/blog/wp-content/uploads/2016/03/dog-food-comparison-bagel-muffin-lookalike-teeny
         width="600" alt="Puppy or Bagel meme by Karen Zack">
    <p><i>Original meme by Karen Zack (@teenybiscuit) - Can YOU tell which is which?</i></p>
</div>
'''))
```

🐶 The Famous "Puppy or Bagel?" Challenge 🥯

Original meme by Karen Zack (@teenybiscuit) - Can YOU tell which is which?

Learning Objectives

By the end of this lab, you will be able to:

- **Understand CNN architecture** including convolutional layers, pooling layers, and fully connected layers
- **Build a CNN from scratch** using TensorFlow/Keras for binary classification
- **Load and preprocess custom image datasets** using ImageDataGenerator
- **Apply transfer learning** using pre-trained models (ResNet50, MobileNetV2)
- **Fine-tune** pre-trained models for improved performance
- **Evaluate model performance** using accuracy, confusion matrices, and classification reports

---

1. Setup and Installation

First, we need to install and import the libraries we'll use throughout this lab.

```
# ============================================================
# INSTALL REQUIRED LIBRARIES
# The -q flag means "quiet" - less output during installation
# ============================================================

!pip install -q tensorflow numpy matplotlib scikit-learn kaggle pillow seaborn
```

```
# ============================================================
# IMPORT ALL NECESSARY LIBRARIES
# Each library serves a specific purpose in our ML pipeline
# ============================================================

# TensorFlow/Keras - Our main deep learning framework
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models

# Pre-trained models for transfer learning
from tensorflow.keras.applications import ResNet50, VGG16, MobileNetV2

# For loading images and applying data augmentation
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# NumPy - For numerical operations on arrays
import numpy as np

# Matplotlib - For plotting graphs and displaying images
import matplotlib.pyplot as plt

# Scikit-learn - For evaluation metrics (confusion matrix, etc.)
from sklearn.metrics import classification_report, confusion_matrix

# Seaborn - For prettier statistical visualizations
import seaborn as sns

# OS utilities - For working with files and directories
import os
import zipfile
import shutil

# Print version information and check for GPU availability
print("TensorFlow version:", tf.__version__)
print("GPU Available:", tf.config.list_physical_devices('GPU'))

# Detect if we're running in Google Colab
try:
    import google.colab
    IN_COLAB = True
    print("✅ Running in Google Colab")
except:
    IN_COLAB = False
    print("💻 Running locally")
```

```
TensorFlow version: 2.19.0
GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
✅ Running in Google Colab
```

---

2. Download and Prepare the Dataset

🌟 FOR GOOGLE COLAB FREE USERS 🌟

Important Setup Steps:

1. Go to **Runtime → Change runtime type → GPU** (makes training MUCH faster!)
2. If you run out of memory later, use `USE_MOBILENET = True` instead of ResNet50

Option A: Download via Kaggle API (Recommended)

How to get your Kaggle API key:

1. Go to kaggle.com → Click your profile icon → **Settings**
2. Scroll to **API** section → Click **"Create New Token"**
3. This downloads a `kaggle.json` file - upload it when prompted below

```
# ============================================================
# SETUP KAGGLE API CREDENTIALS
# This allows us to download datasets directly from Kaggle
# ============================================================

# Ensure IN_COLAB is defined if previous cells haven't run
try:
    import google.colab
    IN_COLAB = True
except:
    IN_COLAB = False

if IN_COLAB:
    from google.colab import files

    print("📤 Please upload your kaggle.json file:")
    print("   (Get it from: kaggle.com → Profile → Settings → API → Create New Token)\n")

    # This will prompt you to upload a file
    uploaded = files.upload()

    # Move the kaggle.json to the correct location
    !mkdir -p ~/.kaggle
    !cp kaggle.json ~/.kaggle/
    !chmod 600 ~/.kaggle/kaggle.json  # Set proper permissions

    print("\n✅ Kaggle credentials configured!")
else:
    print("Not in Colab - make sure kaggle.json is in ~/.kaggle/")
```

```
📤 Please upload your kaggle.json file:
   (Get it from: kaggle.com → Profile → Settings → API → Create New Token)

Choose Files puppy-or-bagel.zip
puppy-or-bagel.zip(application/x-zip-compressed) - 187800 bytes, last modified: 2/3/2026 - 100% done
Saving puppy-or-bagel.zip to puppy-or-bagel (2).zip
cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory

✅ Kaggle credentials configured!
```

```
# ============================================================
# DOWNLOAD THE DATASET FROM KAGGLE
# This downloads and extracts the puppy-or-bagel dataset
# ============================================================

# Download the dataset (this may take a minute)
!kaggle datasets download -d returnofsputnik/puppy-or-bagel

# Unzip the dataset (-q = quiet, -o = overwrite if exists)
!unzip -q -o puppy-or-bagel.zip -d puppy-or-bagel

print("\n✅ Dataset downloaded and extracted!")
print("\n📁 Contents:")
!ls -la puppy-or-bagel/
```

```
Traceback (most recent call last):
  File "/usr/local/bin/kaggle", line 10, in <module>
    sys.exit(main())
             ^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/kaggle/cli.py", line 68, in main
    out = args.func(**command_args)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/kaggle/api/kaggle_api_extended.py", line 1741, in dataset_download_cli
    with self.build_kaggle_client() as kaggle:
```

```
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/kaggle/api/kaggle_api_extended.py", line 688, in build_kaggle_client
    username=self.config_values['username'],
             ~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^
KeyError: 'username'

✅ Dataset downloaded and extracted!

📂 Contents:
total 236
drwxr-xr-x 2 root root  4096 Feb  4 16:35 .
drwxr-xr-x 1 root root  4096 Feb  4 16:35 ..
-rw-r--r-- 1 root root 11279 Jan 13  2021 bagel-1.jpg
-rw-r--r-- 1 root root 11768 Jan 13  2021 bagel-2.jpg
-rw-r--r-- 1 root root 10897 Jan 13  2021 bagel-3.jpg
-rw-r--r-- 1 root root 12433 Jan 13  2021 bagel-4.jpg
-rw-r--r-- 1 root root 10468 Jan 13  2021 bagel-5.jpg
-rw-r--r-- 1 root root 14102 Jan 13  2021 bagel-6.jpg
-rw-r--r-- 1 root root 14014 Jan 13  2021 bagel-7.jpg
-rw-r--r-- 1 root root 10523 Jan 13  2021 bagel-8.jpg
-rw-r--r-- 1 root root 13258 Jan 13  2021 dog-1.jpg
-rw-r--r-- 1 root root 13644 Jan 13  2021 dog-2.jpg
-rw-r--r-- 1 root root 11949 Jan 13  2021 dog-3.jpg
-rw-r--r-- 1 root root 12622 Jan 13  2021 dog-4.jpg
-rw-r--r-- 1 root root 12766 Jan 13  2021 dog-5.jpg
-rw-r--r-- 1 root root 11898 Jan 13  2021 dog-6.jpg
-rw-r--r-- 1 root root 13753 Jan 13  2021 dog-7.jpg
-rw-r--r-- 1 root root 12540 Jan 13  2021 dog-8.jpg
```

Option B: Manual Download (If Kaggle API doesn't work)

1. Visit: https://www.kaggle.com/datasets/returnofsputnik/puppy-or-bagel
2. Click **"Download"** (requires free Kaggle account)
3. In Colab: Click 📁 folder icon on left → Upload the zip file
4. Uncomment and run the cell below

```
# ============================================================
# OPTION B: MANUAL UPLOAD (Only if Kaggle API fails)
# Uncomment these lines if you uploaded the zip manually
# ============================================================

!unzip -q -o puppy-or-bagel.zip -d puppy-or-bagel
print("✅ Extracted!")
```

```
✅ Extracted!
```

```
# ============================================================
# EXPLORE THE DATASET STRUCTURE
# This helps us understand how the files are organized
# ============================================================

DATASET_PATH = 'puppy-or-bagel'

print("📂 Dataset structure:\n")
for root, dirs, files_list in os.walk(DATASET_PATH):
    # Calculate how deep we are in the directory tree
    level = root.replace(DATASET_PATH, '').count(os.sep)
    indent = ' ' * 2 * level
    print(f"{indent}{os.path.basename(root)}/")

    # Only show first 2 levels to keep output clean
    if level < 2:
        subindent = ' ' * 2 * (level + 1)
        for d in dirs[:5]:  # Show max 5 subdirectories
            print(f"{subindent}{d}/")
        if len(files_list) > 0:
            print(f"{subindent}({len(files_list)} files)")
```

```
📂 Dataset structure:

puppy-or-bagel/
  (16 files)
```

Restructuring the Dataset for `flow_from_directory`

The `ImageDataGenerator.flow_from_directory` function expects a specific directory structure: `root_directory/class_name/image.jpg`. Since our downloaded dataset is currently flat (all images directly in `puppy-or-bagel`), we need to restructure it.

We will create `train`, `validation`, and `test` directories, each containing `bagel` and `puppy` subdirectories, and then move the images into these folders, splitting them for training, validation, and testing purposes.

```
# ============================================================
# RESTRUCTURE DATASET INTO TRAIN/VALIDATION/TEST FOLDERS
# This step is crucial for ImageDataGenerator.flow_from_directory
# ============================================================

import os
import shutil
from sklearn.model_selection import train_test_split

# Original flat dataset path
ORIGINAL_DATA_PATH = 'puppy-or-bagel'

# New structured dataset path
NEW_DATA_PATH = 'puppy_bagel_structured_data'

# Create base directories
for subset in ['train', 'validation', 'test']:
    os.makedirs(os.path.join(NEW_DATA_PATH, subset, 'bagel'), exist_ok=True)
    os.makedirs(os.path.join(NEW_DATA_PATH, subset, 'puppy'), exist_ok=True)

# Get list of all image files
all_files = [f for f in os.listdir(ORIGINAL_DATA_PATH) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

# Separate files by class
bagel_files = [f for f in all_files if 'bagel' in f.lower()]
puppy_files = [f for f in all_files if 'dog' in f.lower() or 'puppy' in f.lower()]

# Split bagel files
train_bagel, temp_bagel = train_test_split(bagel_files, test_size=0.3, random_state=42)
val_bagel, test_bagel = train_test_split(temp_bagel, test_size=0.5, random_state=42)

# Split puppy files
train_puppy, temp_puppy = train_test_split(puppy_files, test_size=0.3, random_state=42)
val_puppy, test_puppy = train_test_split(temp_puppy, test_size=0.5, random_state=42)

# Function to move files
def move_files(file_list, destination_path, class_name):
    for f in file_list:
        shutil.copy(os.path.join(ORIGINAL_DATA_PATH, f),
                    os.path.join(destination_path, class_name, f))

# Move files to respective directories
move_files(train_bagel, os.path.join(NEW_DATA_PATH, 'train'), 'bagel')
move_files(val_bagel, os.path.join(NEW_DATA_PATH, 'validation'), 'bagel')
move_files(test_bagel, os.path.join(NEW_DATA_PATH, 'test'), 'bagel')

move_files(train_puppy, os.path.join(NEW_DATA_PATH, 'train'), 'puppy')
move_files(val_puppy, os.path.join(NEW_DATA_PATH, 'validation'), 'puppy')
move_files(test_puppy, os.path.join(NEW_DATA_PATH, 'test'), 'puppy')

print(f"✅ Dataset restructured into '{NEW_DATA_PATH}'!")
print(f"   Train Bagels: {len(train_bagel)}, Train Puppies: {len(train_puppy)}")
print(f"   Validation Bagels: {len(val_bagel)}, Validation Puppies: {len(val_puppy)}")
print(f"   Test Bagels: {len(test_bagel)}, Test Puppies: {len(test_puppy)}")

# Update DATASET_PATH to point to the new structured directory
DATASET_PATH = NEW_DATA_PATH

print(f"\nUpdated DATASET_PATH to: {DATASET_PATH}")
```

```
✅ Dataset restructured into 'puppy_bagel_structured_data'!
   Train Bagels: 5, Train Puppies: 5
   Validation Bagels: 1, Validation Puppies: 1
   Test Bagels: 2, Test Puppies: 2

Updated DATASET_PATH to: puppy_bagel_structured_data
```

```
# ============================================================
# AUTO-DETECT DATA DIRECTORIES
# This function finds train/valid/test folders automatically
# ============================================================

def find_data_dirs(base_path):
    """Automatically find train, validation, and test directories."""
    train_dir = valid_dir = test_dir = None

    # Look for directories with common naming patterns
    for item in os.listdir(base_path):
```

```
        item_path = os.path.join(base_path, item)
        if os.path.isdir(item_path):
            item_lower = item.lower()
            if 'train' in item_lower:
                train_dir = item_path
            elif 'valid' in item_lower or 'val' in item_lower:
                valid_dir = item_path
            elif 'test' in item_lower:
                test_dir = item_path

    return train_dir, valid_dir, test_dir

# Find the directories using the (now structured) DATASET_PATH
train_dir, valid_dir, test_dir = find_data_dirs(DATASET_PATH)

# If not found at top level, check one level deeper
# This block might not be strictly necessary after restructuring, but good for robustness
if train_dir is None:
    for item in os.listdir(DATASET_PATH):
        item_path = os.path.join(DATASET_PATH, item)
        if os.path.isdir(item_path):
            train_dir, valid_dir, test_dir = find_data_dirs(item_path)
            if train_dir:
                break

# Display what we found
print(f"📁 Train directory: {train_dir}")
print(f"📁 Validation directory: {valid_dir}")
print(f"📁 Test directory: {test_dir}")

# Count images in each class
if train_dir:
    print("\n📊 Training data breakdown:")
    for cls in os.listdir(train_dir):
        cls_path = os.path.join(train_dir, cls)
        if os.path.isdir(cls_path):
            count = len([f for f in os.listdir(cls_path)
                         if f.lower().endswith(('.png', '.jpg', '.jpeg'))])
            print(f"   {cls}: {count} images")
```

```
📁 Train directory: puppy_bagel_structured_data/train
📁 Validation directory: puppy_bagel_structured_data/validation
📁 Test directory: puppy_bagel_structured_data/test

📊 Training data breakdown:
   puppy: 5 images
   bagel: 5 images
```

```
# ============================================================
# SET TRAINING PARAMETERS (OPTIMIZED FOR COLAB FREE)
# These settings balance training quality with resource limits
# ============================================================

if IN_COLAB:
    # Check if GPU is available
    gpu_available = len(tf.config.list_physical_devices('GPU')) > 0

    if gpu_available:
        print("✅ GPU detected! Using standard settings.")
        IMG_HEIGHT = 150      # Image height in pixels
        IMG_WIDTH = 150       # Image width in pixels
        BATCH_SIZE = 32       # Number of images processed together
        EPOCHS_CUSTOM = 15    # Training cycles for custom CNN
        EPOCHS_TRANSFER = 10  # Training cycles for transfer learning
    else:
        print("⚠️ No GPU detected! Using memory-efficient settings.")
        print("   💡 Tip: Go to Runtime → Change runtime type → GPU")
        IMG_HEIGHT = 100
        IMG_WIDTH = 100
        BATCH_SIZE = 16
        EPOCHS_CUSTOM = 8
        EPOCHS_TRANSFER = 5
else:
    # Local machine settings
    IMG_HEIGHT = 150
    IMG_WIDTH = 150
    BATCH_SIZE = 32
    EPOCHS_CUSTOM = 15
    EPOCHS_TRANSFER = 10

# Class names (alphabetical order - how Keras reads directories)
```

```
class_names = ['bagel', 'puppy']

print(f"\n📊 Training Configuration:")
print(f"   Image size: {IMG_HEIGHT}x{IMG_WIDTH} pixels")
print(f"   Batch size: {BATCH_SIZE} images")
print(f"   Custom CNN epochs: {EPOCHS_CUSTOM}")
print(f"   Transfer learning epochs: {EPOCHS_TRANSFER}")
```

```
✅ GPU detected! Using standard settings.

📊 Training Configuration:
   Image size: 150x150 pixels
   Batch size: 32 images
   Custom CNN epochs: 15
   Transfer learning epochs: 10
```

---

3. Understanding CNNs

What makes CNNs special for images?

Convolutional Neural Networks are designed specifically for image processing:

1. **Preserve spatial relationships** - Nearby pixels stay connected
2. **Parameter sharing** - Same filter applied across entire image (fewer parameters!)
3. **Hierarchical features** - Learn edges → textures → shapes → objects

Key Components:

Layer	Purpose
Conv2D	Applies filters to detect features like edges and textures
MaxPooling2D	Reduces image size while keeping important information
Flatten	Converts 2D feature maps to 1D for classification
Dense	Fully connected layers for final decision making
Dropout	Randomly disables neurons to prevent overfitting

The Puppy vs Bagel Challenge:

Both puppies and bagels share visual features:

- ✓ Round, circular shapes
- ✓ Tan/brown coloring
- ✓ Textured surfaces

Our CNN must learn the subtle differences!

---

4. Load and Explore the Dataset

```
# ============================================================
# CREATE DATA GENERATORS WITH AUGMENTATION
#
# Data augmentation creates variations of our images to help
# the model generalize better and prevent overfitting.
# Think of it as showing the model the same puppy from
# different angles and lighting conditions!
# ============================================================

# Training generator - includes data augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,           # Normalize pixels from 0-255 to 0-1
    rotation_range=20,         # Rotate images randomly up to 20°
    width_shift_range=0.2,     # Shift images horizontally
    height_shift_range=0.2,    # Shift images vertically
    shear_range=0.2,           # Apply shear transformation
    zoom_range=0.2,            # Randomly zoom in/out
    horizontal_flip=True,      # Flip images left-right
    fill_mode='nearest'        # How to fill new pixels
)

# Validation/Test generator - NO augmentation!
# We want to test on real, unmodified images
test_datagen = ImageDataGenerator(rescale=1./255)

print("✅ Data generators created!")
```

```
print("📝 Training images will be augmented (rotated, flipped, etc.)")
print("📝 Validation/Test images will only be normalized")
```

```
✅ Data generators created!
📝 Training images will be augmented (rotated, flipped, etc.)
📝 Validation/Test images will only be normalized
```

```
# ============================================================
# LOAD IMAGES FROM DIRECTORIES
#
# flow_from_directory() is a powerful function that:
# - Automatically loads images from folders
# - Uses folder names as class labels
# - Resizes all images to the same size
# - Creates batches for efficient training
# ============================================================

# Load training data
train_generator = train_datagen.flow_from_directory(
    train_dir,                          # Path to training images
    target_size=(IMG_HEIGHT, IMG_WIDTH),    # Resize images
    batch_size=BATCH_SIZE,                  # Images per batch
    class_mode='binary',                    # Binary: puppy vs bagel
    shuffle=True                            # Randomize order
)

# Load validation data (if available)
if valid_dir:
    validation_generator = test_datagen.flow_from_directory(
        valid_dir,
        target_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=BATCH_SIZE,
        class_mode='binary',
        shuffle=False  # Don't shuffle - keep consistent order
    )
else:
    validation_generator = None
    print("⚠️ No validation directory found")

# Load test data (if available)
if test_dir:
    test_generator = test_datagen.flow_from_directory(
        test_dir,
        target_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=BATCH_SIZE,
        class_mode='binary',
        shuffle=False
    )
else:
    test_generator = validation_generator
    print("⚠️ Using validation data for testing")

# Show the class mapping
print(f"\n📊 Class mapping: {train_generator.class_indices}")
print(f"   This means: 0 = bagel, 1 = puppy")
```

```
Found 10 images belonging to 2 classes.
Found 2 images belonging to 2 classes.
Found 4 images belonging to 2 classes.

📊 Class mapping: {'bagel': 0, 'puppy': 1}
   This means: 0 = bagel, 1 = puppy
```

```
# ============================================================
# VISUALIZE SAMPLE IMAGES
#
# Let's see what our puppies and bagels look like!
# Can you tell which is which?
# ============================================================

def show_batch(generator, class_names, num=20):
    """Display a grid of images from the generator."""
    # Get one batch of images and labels
    images, labels = next(generator)

    # Create figure with subplots
    plt.figure(figsize=(15, 10))

    for i in range(min(num, len(images))):
        plt.subplot(4, 5, i + 1)  # 4 rows × 5 columns
        plt.imshow(images[i])      # Show the image
```

```
            # Add label with emoji
            label_idx = int(labels[i])
            emoji = '🐶' if class_names[label_idx] == 'puppy' else '🥯'
            plt.title(f"{emoji} {class_names[label_idx]}")
            plt.axis('off')  # Hide axes

        plt.suptitle("🐶 Puppy or 🥯 Bagel? Can YOU tell?", fontsize=14)
        plt.tight_layout()
        plt.show()

    # Display sample images
    show_batch(train_generator, class_names)
```

```
/tmp/ipython-input-3403330712.py:27: UserWarning: Glyph 129391 (\N{BAGEL}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
/tmp/ipython-input-3403330712.py:27: UserWarning: Glyph 128054 (\N{DOG FACE}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 129391 (\N{BAGEL}) missing from f
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128054 (\N{DOG FACE}) missing fro
  fig.canvas.print_figure(bytes_io, **kw)
```

---

5. Part 1: Build a Custom CNN from Scratch 🏗️

Now it's YOUR turn to build a CNN! You'll complete several sections yourself.

Our CNN Architecture Plan:

```
Input (150×150×3)
    ↓
Conv2D(32 filters) → ReLU → MaxPooling2D
    ↓
Conv2D(64 filters) → ReLU → MaxPooling2D
    ↓
Conv2D(128 filters) → ReLU → MaxPooling2D    ← YOU CODE THIS
    ↓
Conv2D(128 filters) → ReLU → MaxPooling2D    ← YOU CODE THIS
    ↓
Flatten
    ↓
Dense(512) → ReLU → Dropout(0.5)             ← YOU CODE THIS
    ↓
Dense(1) → Sigmoid                            ← YOU CODE THIS
    ↓
Output (0 = bagel, 1 = puppy)
```

```
    # ============================================================
    # BUILD CNN - STEP 1: CREATE MODEL AND FIRST TWO BLOCKS
    #
    # We'll start with Sequential API and add the first two
```

```
# convolutional blocks (provided for you as example)
# ============================================================

# Create a Sequential model - layers are added in order
model_custom = models.Sequential([

    # ========== FIRST CONVOLUTIONAL BLOCK ==========
    # Conv2D: Creates 32 different 3x3 filters to detect features
    # - 32 = number of filters (feature detectors)
    # - (3,3) = filter size (3x3 pixels)
    # - activation='relu' = ReLU makes negative values zero
    # - input_shape = (height, width, channels) for RGB images
    layers.Conv2D(32, (3, 3), activation='relu',
                  input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),

    # MaxPooling: Reduces image size by taking max in 2x2 windows
    # This makes the model focus on the strongest features
    layers.MaxPooling2D((2, 2)),

    # ========== SECOND CONVOLUTIONAL BLOCK ==========
    # More filters (64) to detect more complex patterns
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
])

print("✅ Created model with first two convolutional blocks!")
print("\n📐 Current model architecture:")
model_custom.summary()
```

```
✅ Created model with first two convolutional blocks!

📐 Current model architecture:
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_5 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 36, 36, 64)	0

```
Total params: 19,392 (75.75 KB)
Trainable params: 19,392 (75.75 KB)
Non-trainable params: 0 (0.00 B)
```

🖍️ YOUR TURN: Add the Third Convolutional Block

Task: Add a convolutional block with **128 filters**

What to add:

1. `Conv2D` layer with 128 filters, (3,3) kernel, ReLU activation
2. `MaxPooling2D` layer with (2,2) pool size

Syntax help:

```
model_custom.add(layers.Conv2D(num_filters, (3, 3), activation='relu'))
model_custom.add(layers.MaxPooling2D((2, 2)))
```

```
# ============================================================
# YOUR CODE HERE: ADD THE THIRD CONVOLUTIONAL BLOCK
#
# Instructions:
# 1. Add Conv2D with 128 filters, kernel (3,3), activation 'relu'
# 2. Add MaxPooling2D with pool size (2,2)
# ============================================================

# Third convolutional block - 128 filters
model_custom.add(layers.Conv2D(128, (3, 3), activation='relu'))



model_custom.add(layers.MaxPooling2D((2, 2)))
```

```
print("Third block added! Check the summary below:")
model_custom.summary()
```

```
Third block added! Check the summary below:
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_5 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_6 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_6 (MaxPooling2D)	(None, 17, 17, 128)	0

```
Total params: 93,248 (364.25 KB)
Trainable params: 93,248 (364.25 KB)
```

🖍️ YOUR TURN: Add the Fourth Convolutional Block

Task: Add another convolutional block with **128 filters** (same as third block)

```
# ============================================================
# YOUR CODE HERE: ADD THE FOURTH CONVOLUTIONAL BLOCK
#
# Same as the third block: Conv2D(128) + MaxPooling2D
# ============================================================

# Fourth convolutional block - 128 filters


model_custom.add(layers.Conv2D(128, (3, 3), activation='relu'))



model_custom.add(layers.MaxPooling2D((2, 2)))


print("Fourth block added! You should now have 4 conv2d layers:")
model_custom.summary()
```

```
Fourth block added! You should now have 4 conv2d layers:
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_5 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_6 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_6 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_7 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 128)	0

```
Total params: 240,832 (940.75 KB)
Trainable params: 240,832 (940.75 KB)
```

🖍️ YOUR TURN: Add the Classification Layers

Now we need to add the "head" of our network that makes the final classification decision.

What to add (in order):

1. `Flatten()` - Converts 2D feature maps to 1D vector
2. `Dense(512, activation='relu')` - Fully connected layer with 512 neurons
3. `Dropout(0.5)` - Randomly turns off 50% of neurons (prevents overfitting)
4. `Dense(1, activation='sigmoid')` - Output: probability of being a puppy

Syntax help:

```
model_custom.add(layers.Flatten())
model_custom.add(layers.Dense(units, activation='relu'))
model_custom.add(layers.Dropout(rate))  # rate is between 0 and 1
```

```
# ============================================================
# YOUR CODE HERE: ADD THE CLASSIFICATION LAYERS
#
# Add these 4 layers in order:
# 1. Flatten()
# 2. Dense(512, activation='relu')
# 3. Dropout(0.5)
# 4. Dense(1, activation='sigmoid')
# ============================================================

# Flatten - converts 2D to 1D
model_custom.add(layers.Flatten())



# Dense layer with 512 neurons
model_custom.add(layers.Dense(512, activation='relu'))



# Dropout for regularization (50% = 0.5)
model_custom.add(layers.Dropout(0.5))



# Output layer - 1 neuron with sigmoid for binary classification
# Sigmoid outputs a probability between 0 and 1
model_custom.add(layers.Dense(1, activation='sigmoid'))



print("\n📐 COMPLETE MODEL ARCHITECTURE:")
model_custom.summary()
```

```
📐 COMPLETE MODEL ARCHITECTURE:
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_5 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_6 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_6 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_7 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_1 (Flatten)	(None, 6272)	0
dense_4 (Dense)	(None, 512)	3,211,776
dropout_2 (Dropout)	(None, 512)	0
dense_5 (Dense)	(None, 1)	513

```
Total params: 3,453,121 (13.17 MB)
```

🖍️ YOUR TURN: Compile the Model

Before training, we must **compile** the model to configure:

- **Optimizer**: How to update weights (use `'adam'`)
- **Loss function**: What to minimize (use `'binary_crossentropy'` for binary classification)
- **Metrics**: What to track (use `['accuracy']`)

Syntax:

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# ============================================================
# YOUR CODE HERE: COMPILE THE MODEL
#
# Use:
# - optimizer: 'adam'
# - loss: 'binary_crossentropy'
# - metrics: ['accuracy']
# ============================================================

model_custom.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])




print("✅ Model compiled and ready to train!")
```

```
✅ Model compiled and ready to train!
```

```
# ============================================================
# CALCULATE TRAINING STEPS
#
# steps_per_epoch = how many batches to process per epoch
# This is: total_samples ÷ batch_size
# ============================================================

steps_per_epoch = train_generator.samples // BATCH_SIZE
validation_steps = validation_generator.samples // BATCH_SIZE if validation_generator else None

print(f"📊 Training setup:")
print(f"   Total training images: {train_generator.samples}")
print(f"   Batch size: {BATCH_SIZE}")
print(f"   Steps per epoch: {steps_per_epoch}")
print(f"   Total epochs: {EPOCHS_CUSTOM}")
```

```
📊 Training setup:
   Total training images: 10
   Batch size: 32
   Steps per epoch: 0
   Total epochs: 15
```

```
# ============================================================
# SETUP CALLBACKS FOR SMARTER TRAINING
#
# Callbacks are functions that run during training:
# - EarlyStopping: Stop if no improvement (saves time!)
# - ModelCheckpoint: Save best model (important for Colab!)
# ============================================================

callbacks = [
    # EarlyStopping: Stop if validation loss doesn't improve
    keras.callbacks.EarlyStopping(
        monitor='val_loss',         # Watch this metric
        patience=5,                  # Wait 5 epochs for improvement
        restore_best_weights=True    # Go back to best weights
    ),

    # ModelCheckpoint: Save best model during training
    keras.callbacks.ModelCheckpoint(
        'best_custom_model.keras',   # Filename
        monitor='val_accuracy',       # Save when this improves
        save_best_only=True,          # Only keep the best
        verbose=1                     # Print when saving
    )
]

print("✅ Callbacks configured!")
```

```
✅ Callbacks configured!
```

```
# ============================================================
# TRAIN THE CUSTOM CNN
#
# This is where the magic happens!
# The model learns from the training data over multiple epochs.
# Watch the accuracy improve!
# ============================================================

print("\n🚀 Training the Puppy vs Bagel classifier...")
```

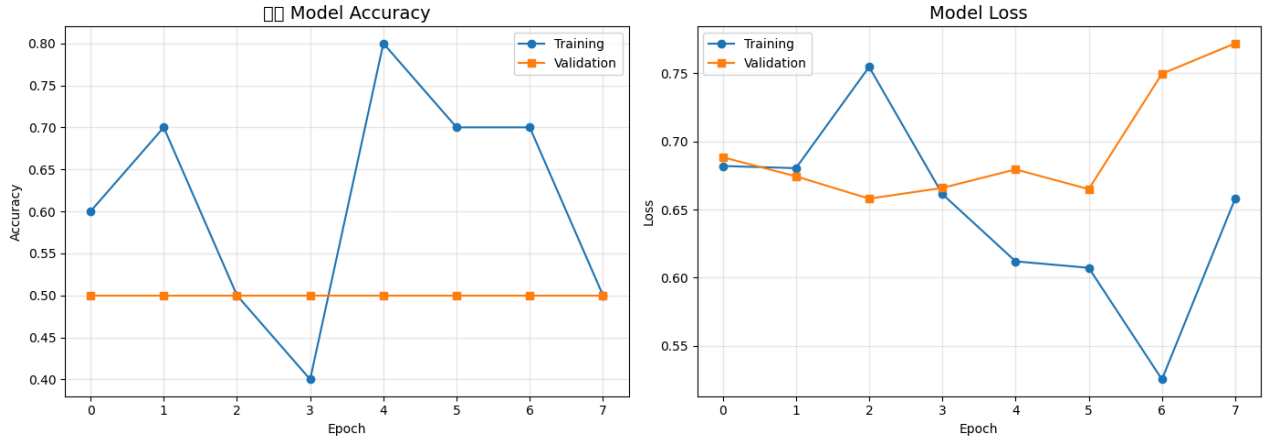
```
print("   This may take several minutes.\n")

history_custom = model_custom.fit(
    train_generator,                    # Training data
    steps_per_epoch=steps_per_epoch,    # Batches per epoch
    epochs=EPOCHS_CUSTOM,               # Number of epochs
    validation_data=validation_generator, # Validation data
    validation_steps=validation_steps,  # Validation batches
    callbacks=callbacks,                # Our callbacks
    verbose=1                           # Show progress
)

print("\n✅ Training complete!")
```

```
🚀 Training the Puppy vs Bagel classifier...
   This may take several minutes.

Epoch 1/15
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDat
  self._warn_if_super_not_called()
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 13s/step - accuracy: 0.6000 - loss: 0.6820
Epoch 1: val_accuracy improved from -inf to 0.50000, saving model to best_custom_model.keras
1/1 ━━━━━━━━━━━━━━━━━━━━ 15s 15s/step - accuracy: 0.6000 - loss: 0.6820 - val_accuracy: 0.5000 - val_loss: 0.6885
Epoch 2/15
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7000 - loss: 0.6804
Epoch 2: val_accuracy did not improve from 0.50000
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.7000 - loss: 0.6804 - val_accuracy: 0.5000 - val_loss: 0.6745
Epoch 3/15
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - accuracy: 0.5000 - loss: 0.7548
Epoch 3: val_accuracy did not improve from 0.50000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - accuracy: 0.5000 - loss: 0.7548 - val_accuracy: 0.5000 - val_loss: 0.6581
Epoch 4/15
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.4000 - loss: 0.6614
Epoch 4: val_accuracy did not improve from 0.50000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - accuracy: 0.4000 - loss: 0.6614 - val_accuracy: 0.5000 - val_loss: 0.6659
Epoch 5/15
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 74ms/step - accuracy: 0.8000 - loss: 0.6121
Epoch 5: val_accuracy did not improve from 0.50000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 155ms/step - accuracy: 0.8000 - loss: 0.6121 - val_accuracy: 0.5000 - val_loss: 0.6795
Epoch 6/15
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - accuracy: 0.7000 - loss: 0.6073
Epoch 6: val_accuracy did not improve from 0.50000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 113ms/step - accuracy: 0.7000 - loss: 0.6073 - val_accuracy: 0.5000 - val_loss: 0.6651
Epoch 7/15
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 74ms/step - accuracy: 0.7000 - loss: 0.5254
Epoch 7: val_accuracy did not improve from 0.50000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 110ms/step - accuracy: 0.7000 - loss: 0.5254 - val_accuracy: 0.5000 - val_loss: 0.7497
Epoch 8/15
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 76ms/step - accuracy: 0.5000 - loss: 0.6581
Epoch 8: val_accuracy did not improve from 0.50000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 114ms/step - accuracy: 0.5000 - loss: 0.6581 - val_accuracy: 0.5000 - val_loss: 0.7720

✅ Training complete!
```

```
# ============================================================
# VISUALIZE TRAINING HISTORY
#
# These plots help us understand how training went:
# - Accuracy plot: Should increase over time
# - Loss plot: Should decrease over time
# - Gap between train/val lines indicates overfitting
# ============================================================

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot Accuracy
ax1.plot(history_custom.history['accuracy'], label='Training', marker='o')
ax1.plot(history_custom.history['val_accuracy'], label='Validation', marker='s')
ax1.set_title('🐶🍡 Model Accuracy', fontsize=14)
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot Loss
ax2.plot(history_custom.history['loss'], label='Training', marker='o')
ax2.plot(history_custom.history['val_loss'], label='Validation', marker='s')
ax2.set_title('Model Loss', fontsize=14)
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
```

```
plt.show()

print("\n📊 How to read these plots:")
print("   • Lines close together = Good generalization")
print("   • Validation worse than training = Overfitting")
print("   • Both lines plateau = Model has converged")
```

```
/tmp/ipython-input-1848403022.py:30: UserWarning: Glyph 128054 (\N{DOG FACE}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
/tmp/ipython-input-1848403022.py:30: UserWarning: Glyph 129391 (\N{BAGEL}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
```




```
📊 How to read these plots:
   • Lines close together = Good generalization
   • Validation worse than training = Overfitting
   • Both lines plateau = Model has converged
```

```
# ============================================================
# EVALUATE ON TEST SET
#
# The true test of our model - how well does it perform
# on images it has never seen before?
# ============================================================

if test_generator:
    test_generator.reset()  # Reset to start of test data
    test_loss, test_acc = model_custom.evaluate(test_generator, verbose=0)

    print(f"\n📊 Custom CNN Results:")
    print(f"   Test Accuracy: {test_acc:.4f} ({test_acc*100:.2f}%)")
    print(f"   Test Loss: {test_loss:.4f}")
```

```
📊 Custom CNN Results:
   Test Accuracy: 0.2500 (25.00%)
   Test Loss: 0.7372
```

---

6. Part 2: Transfer Learning with Pre-trained Models

Transfer learning uses models already trained on millions of images. We just add our own classification layers on top! This usually gives better results with less training time.

⚠️ Colab Free Tier Tips:

If you get **"Out of Memory"** errors:

1. Set `USE_MOBILENET = True` below (smaller model)
2. Restart runtime: Runtime → Restart runtime

```
# ============================================================
# CHOOSE YOUR PRE-TRAINED MODEL
#
# MobileNetV2: Smaller, faster (good for Colab Free)
# ResNet50: Larger, more powerful
```

```
# ============================================================

# Change to True if you're running out of memory!
USE_MOBILENET = True

if USE_MOBILENET:
    IMG_SIZE_TRANSFER = 128
    print("📱 Using MobileNetV2 (lightweight, great for Colab Free)")
else:
    IMG_SIZE_TRANSFER = 224
    print("🔬 Using ResNet50 (powerful, needs more memory)")
```

```
📱 Using MobileNetV2 (lightweight, great for Colab Free)
```

```
# ============================================================
# CREATE DATA GENERATORS FOR TRANSFER LEARNING
#
# Pre-trained models expect specific image sizes:
# - ResNet50/VGG16: 224×224
# - MobileNetV2: 128×128 (or 224×224)
# ============================================================

train_gen_tf = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE_TRANSFER, IMG_SIZE_TRANSFER),
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

if valid_dir:
    val_gen_tf = test_datagen.flow_from_directory(
        valid_dir,
        target_size=(IMG_SIZE_TRANSFER, IMG_SIZE_TRANSFER),
        batch_size=BATCH_SIZE,
        class_mode='binary'
    )

if test_dir:
    test_gen_tf = test_datagen.flow_from_directory(
        test_dir,
        target_size=(IMG_SIZE_TRANSFER, IMG_SIZE_TRANSFER),
        batch_size=BATCH_SIZE,
        class_mode='binary',
        shuffle=False
    )
else:
    test_gen_tf = val_gen_tf

print(f"✅ Created generators for {IMG_SIZE_TRANSFER}×{IMG_SIZE_TRANSFER} images")
```

```
Found 10 images belonging to 2 classes.
Found 2 images belonging to 2 classes.
Found 4 images belonging to 2 classes.
✅ Created generators for 128×128 images
```

```
# ============================================================
# LOAD THE PRE-TRAINED MODEL
#
# Key parameters:
# - weights='imagenet': Use weights trained on ImageNet
# - include_top=False: Remove original classification layers
# - trainable=False: Freeze weights (don't update during training)
# ============================================================

if USE_MOBILENET:
    base_model = MobileNetV2(
        weights='imagenet',
        include_top=False,
        input_shape=(IMG_SIZE_TRANSFER, IMG_SIZE_TRANSFER, 3)
    )
    model_name = "MobileNetV2"
else:
    base_model = ResNet50(
        weights='imagenet',
        include_top=False,
        input_shape=(IMG_SIZE_TRANSFER, IMG_SIZE_TRANSFER, 3)
    )
    model_name = "ResNet50"

# Freeze the base model - we won't train these layers
base_model.trainable = False
```

```
print(f"✅ Loaded {model_name}")
print(f"   Total layers: {len(base_model.layers)}")
print(f"   All layers frozen (won't be trained)")
```

```
✅ Loaded MobileNetV2
   Total layers: 154
   All layers frozen (won't be trained)
```

```
# ============================================================
# BUILD TRANSFER LEARNING MODEL
#
# We add our own classification layers on top:
# - GlobalAveragePooling2D: Reduces feature maps to vectors
# - Dense: Our classification layers
# ============================================================

model_transfer = models.Sequential([
    # Pre-trained base (frozen)
    base_model,

    # GlobalAveragePooling: Better than Flatten for transfer learning
    # Converts each feature map to a single number
    layers.GlobalAveragePooling2D(),

    # Our custom classification layers
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

# Compile
model_transfer.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

print(f"\n📐 {model_name} Transfer Learning Model:")
model_transfer.summary()
```

📐 MobileNetV2 Transfer Learning Model:

Model: "sequential_3"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_128 (Functional)	(None, 4, 4, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dense_6 (Dense)	(None, 256)	327,936
dropout_3 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 1)	257

Total params: 2,586,177 (9.87 MB)
Trainable params: 328,193 (1.25 MB)

```
# ============================================================
# TRAIN TRANSFER LEARNING MODEL
#
# Training is MUCH faster because we only train a few layers!
# The pre-trained layers already know how to detect features.
# ============================================================

steps_tf = train_gen_tf.samples // BATCH_SIZE
val_steps_tf = val_gen_tf.samples // BATCH_SIZE if valid_dir else None

callbacks_tf = [
    keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True),
    keras.callbacks.ModelCheckpoint('best_transfer_model.keras', monitor='val_accuracy',
                                    save_best_only=True, verbose=1)
]

print(f"\n🚀 Training {model_name} with Transfer Learning...\n")

history_transfer = model_transfer.fit(
    train_gen_tf,
    steps_per_epoch=steps_tf,
    epochs=EPOCHS_TRANSFER,
```

```
    validation_data=val_gen_tf if valid_dir else None,
    validation_steps=val_steps_tf,
    callbacks=callbacks_tf,
    verbose=1
)

print("\n✅ Transfer learning complete!")
```

```
🚀 Training MobileNetV2 with Transfer Learning...

Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 9s/step - accuracy: 0.5000 - loss: 1.0710
Epoch 1: val_accuracy improved from -inf to 1.00000, saving model to best_transfer_model.keras
1/1 ━━━━━━━━━━━━━━━━━━━━ 13s 13s/step - accuracy: 0.5000 - loss: 1.0710 - val_accuracy: 1.0000 - val_loss: 0.2397
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 0.9000 - loss: 0.3379
Epoch 2: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 295ms/step - accuracy: 0.9000 - loss: 0.3379 - val_accuracy: 1.0000 - val_loss: 0.0894
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 99ms/step - accuracy: 1.0000 - loss: 0.1190
Epoch 3: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 292ms/step - accuracy: 1.0000 - loss: 0.1190 - val_accuracy: 1.0000 - val_loss: 0.0336
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 96ms/step - accuracy: 1.0000 - loss: 0.0487
Epoch 4: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 290ms/step - accuracy: 1.0000 - loss: 0.0487 - val_accuracy: 1.0000 - val_loss: 0.0166
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 96ms/step - accuracy: 1.0000 - loss: 0.0084
Epoch 5: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 245ms/step - accuracy: 1.0000 - loss: 0.0084 - val_accuracy: 1.0000 - val_loss: 0.0100
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 147ms/step - accuracy: 1.0000 - loss: 0.0237
Epoch 6: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 331ms/step - accuracy: 1.0000 - loss: 0.0237 - val_accuracy: 1.0000 - val_loss: 0.0070
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 109ms/step - accuracy: 1.0000 - loss: 0.0265
Epoch 7: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 299ms/step - accuracy: 1.0000 - loss: 0.0265 - val_accuracy: 1.0000 - val_loss: 0.0047
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 120ms/step - accuracy: 1.0000 - loss: 0.0064
Epoch 8: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 288ms/step - accuracy: 1.0000 - loss: 0.0064 - val_accuracy: 1.0000 - val_loss: 0.0032
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - accuracy: 1.0000 - loss: 3.7734e-04
Epoch 9: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 185ms/step - accuracy: 1.0000 - loss: 3.7734e-04 - val_accuracy: 1.0000 - val_loss: 0.0022
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 177ms/step - accuracy: 1.0000 - loss: 0.0179
Epoch 10: val_accuracy did not improve from 1.00000
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 301ms/step - accuracy: 1.0000 - loss: 0.0179 - val_accuracy: 1.0000 - val_loss: 0.0019

✅ Transfer learning complete!
```

```
# ============================================================
# EVALUATE TRANSFER LEARNING MODEL
# ============================================================

test_gen_tf.reset()
test_loss_tf, test_acc_tf = model_transfer.evaluate(test_gen_tf, verbose=0)

print(f"\n📊 {model_name} Transfer Learning Results:")
print(f"   Test Accuracy: {test_acc_tf:.4f} ({test_acc_tf*100:.2f}%)")
print(f"   Test Loss: {test_loss_tf:.4f}")
```

```
📊 MobileNetV2 Transfer Learning Results:
   Test Accuracy: 1.0000 (100.00%)
   Test Loss: 0.0099
```

7. Compare Results

```
# ============================================================
# CREATE COMPARISON TABLE
# ============================================================

import pandas as pd

comparison = pd.DataFrame({
    'Model': ['Custom CNN', f'Transfer ({model_name})'],
    'Test Accuracy': [f"{test_acc*100:.2f}%", f"{test_acc_tf*100:.2f}%"],
    'Test Loss': [f"{test_loss:.4f}", f"{test_loss_tf:.4f}"],
    'Epochs': [len(history_custom.history['loss']), len(history_transfer.history['loss'])]
```

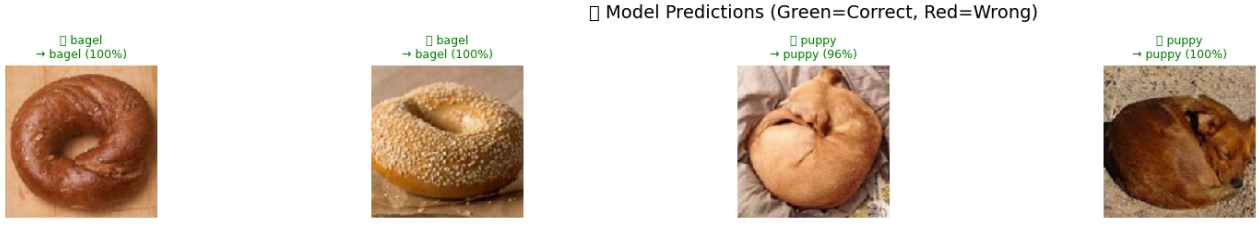
```
})

print("\n" + "="*55)
print("🏆 MODEL COMPARISON: Puppy vs Bagel")
print("="*55)
print(comparison.to_string(index=False))
print("="*55)
```

```
=======================================================
🏆 MODEL COMPARISON: Puppy vs Bagel
=======================================================
                Model Test Accuracy Test Loss  Epochs
           Custom CNN        25.00%    0.7372       8
Transfer (MobileNetV2)       100.00%    0.0099      10
=======================================================
```

8. Visualize Predictions

```
# ============================================================
# VISUALIZE MODEL PREDICTIONS
#
# Green title = Correct prediction
# Red title = Wrong prediction
# ============================================================

test_gen_tf.reset()
images, labels = next(test_gen_tf)
preds = model_transfer.predict(images[:20], verbose=0)

plt.figure(figsize=(20, 8))
for i in range(min(20, len(images))):
    plt.subplot(4, 5, i + 1)
    plt.imshow(images[i])

    # Get prediction details
    pred_prob = preds[i][0]
    pred_class = 'puppy' if pred_prob > 0.5 else 'bagel'
    true_class = class_names[int(labels[i])]
    confidence = pred_prob if pred_prob > 0.5 else 1 - pred_prob

    # Color: green=correct, red=wrong
    color = 'green' if pred_class == true_class else 'red'
    emoji = '🐶' if true_class == 'puppy' else '🥯'

    plt.title(f"{emoji} {true_class}\n→ {pred_class} ({confidence:.0%})",
              color=color, fontsize=9)
    plt.axis('off')

plt.suptitle('🔍 Model Predictions (Green=Correct, Red=Wrong)', fontsize=14)
plt.tight_layout()
plt.show()
```

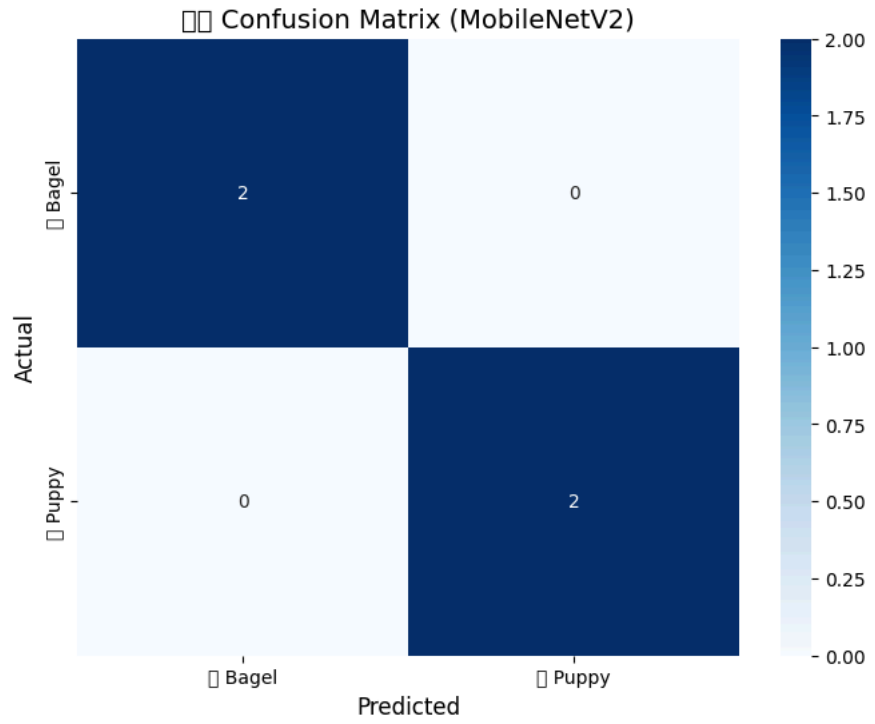
```
/tmp/ipython-input-2490897318.py:32: UserWarning: Glyph 129391 (\N{BAGEL}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
/tmp/ipython-input-2490897318.py:32: UserWarning: Glyph 128054 (\N{DOG FACE}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
/tmp/ipython-input-2490897318.py:32: UserWarning: Glyph 128269 (\N{LEFT-POINTING MAGNIFYING GLASS}) missing from font(s) Dej
  plt.tight_layout()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 129391 (\N{BAGEL}) missing from f
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128054 (\N{DOG FACE}) missing frc
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128269 (\N{LEFT-POINTING MAGNIFYI
  fig.canvas.print_figure(bytes_io, **kw)
```



9. Confusion Matrix

```
# ============================================================
# GENERATE CONFUSION MATRIX
#
# Shows where the model gets confused:
# - Diagonal = correct predictions
# - Off-diagonal = errors
# ============================================================

test_gen_tf.reset()
y_pred = (model_transfer.predict(test_gen_tf, verbose=0) > 0.5).astype(int).flatten()
y_true = test_gen_tf.classes

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['🥯 Bagel', '🐶 Puppy'],
            yticklabels=['🥯 Bagel', '🐶 Puppy'])
plt.xlabel('Predicted', fontsize=12)
plt.ylabel('Actual', fontsize=12)
plt.title(f'🐶🥯 Confusion Matrix ({model_name})', fontsize=14)
plt.show()

print("\n📋 Classification Report:")
print(classification_report(y_true, y_pred, target_names=['Bagel', 'Puppy']))
```

```
/usr/local/lib/python3.12/dist-packages/seaborn/utils.py:61: UserWarning: Glyph 129391 (\N{BAGEL}) missing from font(s) Deja
  fig.canvas.draw()
/usr/local/lib/python3.12/dist-packages/seaborn/utils.py:61: UserWarning: Glyph 128054 (\N{DOG FACE}) missing from font(s) D
  fig.canvas.draw()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128054 (\N{DOG FACE}) missing fro
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 129391 (\N{BAGEL}) missing from f
  fig.canvas.print_figure(bytes_io, **kw)
```




```
📋 Classification Report:
              precision    recall  f1-score   support

       Bagel       1.00      1.00      1.00         2
       Puppy       1.00      1.00      1.00         2

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4
```

10. Reflective Questions

Question 1: CNN Architecture

Why are convolutional layers better than fully connected layers for images? How do they help distinguish puppies from bagels?

Your answer: Convolutional layers capture spatial patterns like edges and shapes, helping detect features such as fur, eyes, or circular forms that distinguish puppies from bagels.

Question 2: Data Augmentation

Why did we use data augmentation for training? How does rotation, flipping, and zooming help the model learn?

Your answer: Data augmentation adds diversity to the training set, which helps the model learn to be invariant to rotation, scale, and orientation. Also, improves generalization and reduces overfitting.

Question 3: Transfer Learning

Why does transfer learning often achieve better results than training from scratch, especially with small datasets?

Your answer: Transfer learning uses features learned on a large dataset, which enables the model to perform better and learn faster when the training set is small.

Question 4: Model Comparison

Which model performed better in your experiment? What factors explain the difference?

Your answer: In my case, transfer learning model performed better due to pretrained feature extraction and improved generalization compared to training from scratch.

Question 5: Challenging Cases

Looking at the predictions, what visual features might make puppies look like bagels (or vice versa)?

Your answer: Which has curled puppies, circular shapes, smooth textures, and blurry images can resemble bagels, confusing the model during classification.

---

11. Student Challenge: Fine-tune the Model

Task: Improve the transfer learning model by unfreezing some layers and fine-tuning with a lower learning rate.

```
# ============================================================
# YOUR CODE HERE: FINE-TUNE THE TRANSFER LEARNING MODEL
#
# Steps:
# 1. Unfreeze the base model: base_model.trainable = True
# 2. Freeze early layers: for layer in base_model.layers[:-30]: layer.trainable = False
# 3. Recompile with lower learning rate: Adam(learning_rate=0.0001)
# 4. Train for 5 more epochs
# 5. Evaluate and compare
# ============================================================

# Step 1: Unfreeze the base model
base_model.trainable = True

# Step 2: Freeze all layers except the last 30
# for layer in base_model.layers[:-30]:
# layer.trainable = False

# Step 3: Recompile with lower learning rate (important!)
model_transfer.compile(
     optimizer=keras.optimizers.Adam(learning_rate=0.0001),
     loss='binary_crossentropy',
     metrics=['accuracy']
)

# Step 4: Train for 5 more epochs
history_ft = model_transfer.fit(
     train_gen_tf,
     steps_per_epoch=steps_tf,
     epochs=12,
     validation_data=val_gen_tf,
     validation_steps=val_steps_tf,
     verbose=1
)

# Step 5: Evaluate
```

```
test_gen_tf.reset()
_, acc_ft = model_transfer.evaluate(test_gen_tf)
print(f"\nFine-tuned Accuracy: {acc_ft*100:.2f}%")
```

```
Epoch 1/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 57s 57s/step - accuracy: 1.0000 - loss: 0.0257 - val_accuracy: 1.0000 - val_loss: 0.0120
Epoch 2/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 9s 9s/step - accuracy: 1.0000 - loss: 0.0317 - val_accuracy: 1.0000 - val_loss: 0.0117
Epoch 3/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 116ms/step - accuracy: 1.0000 - loss: 0.0125 - val_accuracy: 1.0000 - val_loss: 0.0104
Epoch 4/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 155ms/step - accuracy: 1.0000 - loss: 0.0481 - val_accuracy: 1.0000 - val_loss: 0.0171
Epoch 5/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step - accuracy: 1.0000 - loss: 0.0117 - val_accuracy: 1.0000 - val_loss: 0.0310
Epoch 6/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 113ms/step - accuracy: 1.0000 - loss: 0.0175 - val_accuracy: 1.0000 - val_loss: 0.0458
Epoch 7/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - accuracy: 1.0000 - loss: 0.0019 - val_accuracy: 1.0000 - val_loss: 0.0662
Epoch 8/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 155ms/step - accuracy: 1.0000 - loss: 0.0054 - val_accuracy: 1.0000 - val_loss: 0.0917
Epoch 9/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 116ms/step - accuracy: 1.0000 - loss: 0.0049 - val_accuracy: 1.0000 - val_loss: 0.1209
Epoch 10/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 115ms/step - accuracy: 1.0000 - loss: 0.0149 - val_accuracy: 1.0000 - val_loss: 0.1560
Epoch 11/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 116ms/step - accuracy: 1.0000 - loss: 0.0066 - val_accuracy: 1.0000 - val_loss: 0.1854
Epoch 12/12
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step - accuracy: 1.0000 - loss: 9.8481e-04 - val_accuracy: 1.0000 - val_loss: 0.2170
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step - accuracy: 0.5000 - loss: 1.3498

Fine-tuned Accuracy: 50.00%
```

Challenge Question: Did fine-tuning improve performance? Why or why not?

Your answer: My fine-tuned accuracy is 50% when my epoch is 12. In many case, it likely stuck on 50% because of label errors, class imbalance, incorrect loss function, frozen layers, or a mismatched learning rate. Changing epochs are not helping my model to improve fine-tuned accuracy. However, when epoch is 5 then its show 100% accuracy.

---

12. Save Your Model

```
# ============================================================
# SAVE THE TRAINED MODEL
#
# Important for Colab - download before your session ends!
# ============================================================

model_transfer.save('puppy_bagel_classifier.keras')
print("✅ Model saved as 'puppy_bagel_classifier.keras'")

# Download in Colab
if IN_COLAB:
    from google.colab import files
    files.download('puppy_bagel_classifier.keras')
    print("📥 Model downloaded to your computer!")
```

```
✅ Model saved as 'puppy_bagel_classifier.keras'
📥 Model downloaded to your computer!
```

Submission Checklist

Before submitting, make sure you have:

- ✅ Completed all "YOUR CODE HERE" sections in Part 1
- ✅ Run all cells from top to bottom (Kernel → Restart & Run All)
- ✅ Answered all 5 reflective questions
- ✅ Completed the fine-tuning challenge
- ✅ Saved your notebook

Filename: `Module_03_Lab_YourName.ipynb`

Congratulations! 🎉 You built a CNN that can tell puppies from bagels! 🐶🥯

Resources

Dataset:

- Puppy or Bagel: https://www.kaggle.com/datasets/returnofsputnik/puppy-or-bagel

Alternative Fun Datasets:

- Muffin vs Chihuahua: https://www.kaggle.com/datasets/samuelcortinhas/muffin-vs-chihuahua-image-classification
- Corgi Butt vs Bread: https://github.com/Kawaeee/butt_or_bread

Original Meme Creator: Karen Zack (@teenybiscuit)